ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Е.О. Шумова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №7

«Стандартная библиотека C++. Последовательные и ассоциативные контейнеры.

Обобщенные алгоритмы»

по курсу: ОСНОВЫ ПРОГРАММИРОВАНИЯ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2023

1. Условие с вариантом

Реализовать шаблон класса, содержащего: – массив данных (вектор), заполненный случайными числами в диапазоне m1 - m2; – методы, обеспечивающие выполнение действий (согласно варианта) с использованием обобщенных алгоритмов, объектов-функций и предикатов (использование обобщенных алгоритмов обязательно; – обеспечить вывод результатов после выполнения каждого действия; – продемонстрировать работу с объектами класса для различных типов данных.

Вариант №7

- m1=0, m2=10
- вычислить сумму всех элементов
- для каждого элемента вычислить выражение x = k*x + b (k=2, b=5)
- заменить все элементы большие 20 на 10

2. Листинг программы

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
#include <random>

template<typename T>
class GenericArrayOperations {
public:
    GenericArrayOperations(int size, T m1, T m2);
    T CalculateSum();
    void ApplyOperation(T k, T b);
    void ReplaceGreaterThan(T threshold, T replacement);
    void PrintDataWithOriginal();
    void PrintOriginalData();

private:
    std::vector<T> data;
    std::vector<T> originalData; // Вектор для хранения исходных данных
};

template<typename T>
GenericArrayOperations<T>::GenericArrayOperations(int size, T m1, T m2) {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<double> dis(static_cast<double>(m1),
static_cast<double>(m2));

    for (int i = 0; i < size; i++) {
        T value = static_cast<T>(dis(gen));
        data.push_back(value);
        originalData.push_back(value); // Сохраняем исходные данные
    }
}

template<typename T>
T GenericArrayOperations<T>::CalculateSum() {
    T sum = std::accumulate(data.begin(), data.end(), T(0));
```

```
    return sum;
}

template<typename T>
void GenericArrayOperations<T>::ApplyOperation(T k, T b) {
    for (int i = 0; i < data.size(); i++) {
        data[i] = k * data[i] + b;
    }
}

template<typename T>
void GenericArrayOperations<T>::ReplaceGreaterThan(T threshold, T replacement) {
    for (int i = 0; i < data.size(); i++) {
        if (data[i] > threshold) {
            originalData[i] = data[i]; // Сохраняем исходное значение
            data[i] = replacement; // Применяем замену
        }
    }
}

template<typename T>
void GenericArrayOperations<T>::PrintDataWithOriginal() {
    for (int i = 0; i < data.size(); i++) {
        std::cout << data[i] << "(" << originalData[i] << ") "; // Выводим исходное
число рядом с преобразованным
    }
    std::cout << std::endl;
}

template<typename T>
void GenericArrayOperations<T>::PrintOriginalData() {
    for (int i = 0; i < originalData.size(); i++) {
        std::cout << originalData[i] << " "; // Выводим только исходные значения
    }
    std::cout << std::endl;
}

int main() {
    system("chcp 1251 > nul");
    system("color 71");

    int m1_int = 0, m2_int = 10;
    int k_int = 2, b_int = 5;

    GenericArrayOperations<int> intArray(10, m1_int, m2_int);

    std::cout << "Исходные данные (int): \n";
    intArray.PrintOriginalData();

    int sum_int = intArray.CalculateSum();
    std::cout << "Сумма элементов (int): \n" << sum_int << std::endl;

    std::cout << "Результат после операции x = " << k_int << " * x + " << b_int << "
(int): \n";
    intArray.ApplyOperation(k_int, b_int);
    intArray.PrintDataWithOriginal();

    intArray.ReplaceGreaterThan(20, 10);
    std::cout << "Результат после замены элементов больше 20 на 10 (int): \n";
    intArray.PrintDataWithOriginal();

    float m1_float = 0.0f, m2_float = 10.0f;
    float k_float = 2.0f, b_float = 5.0f;

    GenericArrayOperations<float> floatArray(10, m1_float, m2_float);
```

```
    std::cout << "Исходные данные (float): \n";
    floatArray.PrintOriginalData();

    float sum_float = floatArray.CalculateSum();
    std::cout << "Сумма элементов (float): \n" << sum_float << std::endl;

    std::cout << "Результат после операции x = " << k_float << " * x + " << b_float
<< " (float): \n";
    floatArray.ApplyOperation(k_float, b_float);
    floatArray.PrintDataWithOriginal();

    floatArray.ReplaceGreaterThan(20.0f, 10.0f);
    std::cout << "Результат после замены элементов больше 20 на 10 (float): \n";
    floatArray.PrintDataWithOriginal();

    return 0;
}
```

3. Скриншоты с исходными данными и выполнением программы

```
Исходные данные (int):
5 2 4 5 9 7 1 6 2 2
Сумма элементов (int):
43
Результат после операции x = 2 * x + 5 (int):
15(5) 9(2) 13(4) 15(5) 23(9) 19(7) 7(1) 17(6) 9(2) 9(2)
Результат после замены элементов больше 20 на 10 (int):
15(5) 9(2) 13(4) 15(5) 10(23) 19(7) 7(1) 17(6) 9(2) 9(2)
Исходные данные (float):
4.00917 2.31293 3.67534 9.29283 7.2739 3.38006 6.92347 7.35039 0.514676 8.56386
Сумма элементов (float):
53.2966
Результат после операции x = 2 * x + 5 (float):
13.0183(4.00917) 9.62587(2.31293) 12.3507(3.67534) 23.5857(9.29283) 19.5478(7.2739) 11.7601(3.38006) 18.8469(6.92347) 19.7008(7.35039) 6.02935(0.514676) 22.
1277(8.56386)
Результат после замены элементов больше 20 на 10 (float):
13.0183(4.00917) 9.62587(2.31293) 12.3507(3.67534) 10(23.5857) 19.5478(7.2739) 11.7601(3.38006) 18.8469(6.92347) 19.7008(7.35039) 6.02935(0.514676) 10(22.12
77)
```

4. Вывод

Данный код включает определение класса GenericArrayOperations, предоставляющего операции над вектором элементов. В функции main создается объект этого класса для целых чисел, выполняются следующие операции:

- Генерация случайного вектора целых чисел в заданном диапазоне.
- Вычисление суммы элементов вектора.
- Применение операции x = k * x + b ко всем элементам вектора.
- Замена всех элементов вектора, больших 20, на 10.

Результаты операций выводятся на экран.